LOVELY PROFESSIONAL UNIVERSITY

INT333 PROJECT REPORT

AI Resume Analyzer

AI application and integrated DevOps automation lab

Configuration Management and Automation

Submitted by	[Student Name]
Registration number	[Registration Number]
Roll number / section	[Roll Number] / [Section]
Submitted to	[Faculty / Project Guide]
Department	Computer Science and Engineering
Institution	Lovely Professional University
Submission period	October 2026

Table of Contents

Abstract

1. Problem Selection and Relevance

2. Architecture and Configuration-Management Design

3. Terraform Infrastructure as Code (15 marks)

4. Ansible Automation and Playbooks (15 marks)

5. Puppet Implementation (15 marks)

6. Nagios Monitoring (10 marks)

7. Integration and End-to-End Workflow (10 marks)

8. Testing and Technical Evidence (5 marks)

9. Security, Operations, and Limitations

10. Installation and Demonstration

11. Conclusion

Appendix A. Rubric-to-Evidence Mapping

Appendix B. Viva and Demonstration Questions

References

Abstract

Careerly AI Resume Analyzer is a Flask application that compares PDF resume text with job-description text and returns an AI-generated review. This INT333 project also implements a local DevOps lab aligned to the evaluation rubric: Terraform provisions a Docker network and three containers; Ansible connects to the monitoring container, applies Puppet configuration, validates and starts Nagios, and asserts the app health endpoint. Nagios periodically checks the application over the lab network. The Render deployment remains a separate managed-service deployment because Render does not expose its host for configuration management. This report describes the implementation, execution steps, verification, security boundaries, and evidence to capture.

Keywords: Terraform, Ansible, Puppet, Nagios, Docker, GitHub Actions, Flask, CI/CD, configuration management.

1. Problem Selection and Relevance

1.1 Problem statement

Job seekers often have difficulty identifying how a resume aligns with a specific job posting. Reviewing skills and requirements manually is time-consuming and gives little structured feedback. A resume analyzer automates a first-pass comparison and presents an ATS-style score, overlapping skills, gaps, and actionable suggestions.

1.2 Objectives and scope

- Accept text-based PDF resumes and compare resume and role text.
- Display a score, matched/missing skills, strengths, improvement areas, and suggestions.
- Provide job-description cleanup and clearly labeled general drafts for title-only input.
- Demonstrate repeatable provisioning, configuration automation, monitoring, and CI/CD.

This is an advisory prototype, not a commercial ATS or recruitment decision. It does not submit job applications or guarantee a score. Scanned-image PDFs are not OCR-processed. AI output can be inaccurate and should be reviewed.

2. Architecture and Configuration-Management Design

2.1 Architecture overview

The Flask application runs as a Docker web service on Render. A separate disposable local Docker lab demonstrates the required DevOps tools without implying access to Render internals.

```
Browser -> Flask app -> PDF text extraction / identifier redaction -> Groq API
                                    -> structured analysis -> results UI

Local lab:
Terraform -> Docker network + app + monitoring node + Ansible controller
Ansible -> Puppet apply -> Nagios config/services -> app health assertion
```

Layer	Implementation	Responsibility
Application	Flask + HTML/CSS/JavaScript	Input, validation, AI request, result rendering.
Image	Dockerfile	Reproducible app runtime; secrets supplied at runtime.
Provisioning	Terraform Docker provider	Images, isolated network, containers, and outputs.
Orchestration	Ansible controller	Monitoring configuration and post-deploy checks.
Configuration	Puppet module	Nagios plugin, config template, and service state.
Monitoring	Nagios Core + check_http	Poll /health and classify availability/latency.
Hosted deployment	Render Blueprint	Managed app service and /health check.

2.2 Design decisions

- Use a local Docker lab to avoid cloud bills while demonstrating real resources.
- Keep Render separate: it provides no SSH or host-level configuration access.

- Use community.docker.docker so Ansible configures the monitoring container without SSH.
- Puppet owns Nagios desired state; Ansible owns orchestration, validation, and app checks.
- Bind published lab ports only to 127.0.0.1. Keep Terraform state out of Git.

2.3 Repository components

Path	Purpose
terraform/main.tf	Docker resources, network, containers, Ansible provisioner, outputs.
ansible/Dockerfile	Controller image with ansible-core, Docker SDK, and community.docker.
ansible/playbooks/lab-inventory.yml	Monitoring Docker connection and local health-check target.
ansible/playbooks/configure-and-verify.yml	Puppet apply, Nagios validation/services, app health assertion.
puppet/modules/resume_analyzer_monitoring/	Puppet class and ERB template for Nagios objects.
nagios/Dockerfile	Local Nagios/Apache/Puppet runtime image.
render.yaml	Separate Render service and health endpoint declaration.

3. Terraform Infrastructure as Code (15 marks)

3.1 Provider and resources

The root module requires Terraform >= 1.5.0 and pins kreuzwerker/docker to the ~> 3.0 range; .terraform.lock.hcl records the selected provider checksum/version. The provider reads DOCKER_HOST. The module defines a bridge network, builds the application, Ansible, and monitoring images, and creates three containers with stable names and network aliases. App port 5000 and Nagios port 8080 bind to loopback only.

- docker_network.lab creates the isolated resume-analyzer-devops-lab network.
- docker_image resources build the app, Ansible controller, and monitoring images.
- docker_container resources launch the three lab services with network aliases.
- terraform_data.ansible_provisioning runs Ansible after containers exist and re-runs on replacement.
- Terraform destroy removes managed containers/network; locally retained images are not removed.

3.2 Commands

```
$env:DOCKER_HOST = 'npipe:////./pipe/dockerDesktopLinuxEngine'
docker info
terraform -chdir=terraform init
terraform -chdir=terraform fmt -check
terraform -chdir=terraform validate
terraform -chdir=terraform plan
terraform -chdir=terraform apply
terraform -chdir=terraform output
terraform -chdir=terraform destroy

# Linux Docker Engine
export DOCKER_HOST=unix:///var/run/docker.sock
```

Terraform does not automatically resolve Docker Desktop's named context; DOCKER_HOST must point to the engine. Docker Desktop must use Linux containers. Keep state, local plans, tfvars, and credentials private.

4. Ansible Automation and Playbooks (15 marks)

4.1 Controller and inventory

ansible/Dockerfile builds an ephemeral control node and installs ansible-core, Docker SDK, and community.docker. The inventory assigns the monitoring container to the Docker connection plugin and localhost to a local connection. The controller needs the Docker socket for this plugin.

```
all:
  children:
    monitoring:
      hosts:
        resume_analyzer_monitoring:
          ansible_connection: community.docker.docker
          ansible_host: resume-analyzer-monitoring
```

4.2 Playbook and failure behavior

configure-and-verify.yml runs Puppet apply in the monitoring container, validates Nagios with nagios4 -v, and restarts Nagios/Apache. A localhost play retries the app health request up to ten times, requires HTTP 200, and asserts health.json.status == ok. A failed configuration, invalid Nagios config, unavailable Docker API, or unhealthy app fails the Terraform provisioner.

```
ansible-playbook -i /playbooks/lab-inventory.yml \
  /playbooks/configure-and-verify.yml
```

check-render-health.yml remains available for an external Render health probe using SERVICE_URL. It checks the public endpoint only; it is not host configuration management.

5. Puppet Implementation (15 marks)

5.1 Desired state

puppet/modules/resume_analyzer_monitoring/manifests/init.pp defines package, file, and service resources. It ensures nagios-plugins-basic is installed; writes /etc/nagios4/conf.d/resume-analyzer.cfg from an ERB template; and enables/runs nagios4 and apache2. Resource relationships require the package before the config file, and the config file before Nagios.

```
class resume_analyzer_monitoring {
  package { 'nagios-plugins-basic': ensure => installed }
  file { '/etc/nagios4/conf.d/resume-analyzer.cfg':
    ensure => file,
    content => template('resume_analyzer_monitoring/resume-analyzer.cfg.erb'),
    require => Package['nagios-plugins-basic'],
  }
  service { 'nagios4':
    ensure => running, enable => true,
    require => File['/etc/nagios4/conf.d/resume-analyzer.cfg'],
  }
  service { 'apache2': ensure => running, enable => true }
}
```

5.2 Template-managed objects

The ERB template declares a standard check_http command, one app host, and one service. It checks port 5000 and /health, expects HTTP 200, uses 5/10-second warning/critical thresholds, and checks/retries every minute with three maximum attempts. The host resolves on the private Docker network.

```
define command {
  command_name check_resume_analyzer_health
  command_line $USER1$/check_http -H $HOSTADDRESS$ -p 5000 -u /health -e 200 -w 5 -c 10
}
define host {
  use generic-host
  host_name resume-analyzer-app
  address resume-analyzer-app
  max_check_attempts 3
}
define service {
  use generic-service
  host_name resume-analyzer-app
  service_description Flask health endpoint
  check_command check_resume_analyzer_health
}
```

5.3 Apply and validate

Ansible invokes puppet apply after the monitoring container starts. It then validates Nagios and restarts services. Reapplying Puppet should converge to the declared state rather than accumulate manual edits.

```
puppet parser validate puppet/modules/resume_analyzer_monitoring/manifests/init.pp
# Applied by the Ansible playbook inside the monitoring container:
puppet apply --modulepath=/etc/puppetlabs/code/modules \
  /etc/puppetlabs/code/modules/resume_analyzer_monitoring/manifests/init.pp
```

6. Nagios Monitoring (10 marks)

6.1 Check and thresholds

Nagios Core loads the Puppet-generated objects from /etc/nagios4/conf.d. check_http targets the app container's private DNS name on port 5000. The expected status is 200. Response time is OK below warning threshold, WARNING above five seconds, and CRITICAL above ten seconds. Retry settings reduce transient noise; Nagios periodically evaluates the service and displays its state.

6.2 UI and validation

The dashboard is published only on 127.0.0.1:8080/nagios4. Demo credentials are nagiosadmin / devops-demo and are unsuitable for public use. Ansible runs nagios4 -v /etc/nagios4/nagios.cfg before restarting services. After a successful apply and polling interval, capture the app host and health service in OK state; do not claim a screenshot until it exists.

7. Integration and End-to-End Workflow (10 marks)

Step	Tool	Action / result
1	Terraform	Build images and create the network and three containers.
2	Ansible	Terraform invokes the playbook in the controller container.
3	Puppet	Ansible applies the monitoring module through Docker connection.
4	Nagios	Puppet installs plugin, renders host/service config, and manages services.
5	Ansible	Validates Nagios, restarts services, and asserts app health.
6	Nagios	Continues recurring health checks; UI stays loopback-only.
7	Terraform	Destroy removes lab containers and network.

This control flow directly demonstrates Terraform → Ansible → Puppet → Nagios, followed by an application smoke check. The GitHub Actions DevOps lab job is configured to validate and provision the lab on a Linux runner and destroy it afterward.

8. Testing and Technical Evidence (5 marks)

8.1 Application tests

The Python unittest suite covers the health endpoint and no-store header; visible result panels with skill comparison first; the current Groq model; title-only optimization producing a labeled draft; and detailed-description optimization preserving the no-invented-requirements instruction. Run: python -m unittest discover -s tests -v.

8.2 Infrastructure validation and honest status

The five application tests passed. Terraform provider initialization and terraform validate passed with kreuzwerker/docker 3.9.0. A live local apply was attempted but could not reach Docker Desktop: its status remained starting, the docker-desktop WSL distribution was stopped, and Docker API calls returned HTTP 500. Therefore no successful local Ansible/Puppet execution, Nagios state, or live dashboard has been verified in this session. The GitHub Actions integration job is configured but its result must be checked before claiming end-to-end success.

8.3 Evidence to capture

- Terraform apply completion with app_url and nagios_url outputs.
- Ansible recap showing Puppet, Nagios validation, and health assertion successful.
- Nagios dashboard showing resume-analyzer-app / Flask health endpoint in OK state.
- Successful GitHub Actions devops-lab job and cleanup.
- Application screenshot showing matched and missing skills.

Use screenshots from actual successful runs only; do not substitute mock output.

9. Security, Operations, and Limitations

- Mounting /var/run/docker.sock grants broad, effectively root-level control of the Docker host. Use a disposable trusted machine; never expose the controller publicly.
- App and dashboard ports bind to 127.0.0.1. Demo Nagios credentials are local-only and must change before non-local use.
- Terraform state is local and ignored by Git; do not commit it or secrets.
- Render is managed separately; this project does not claim host access there.
- Identifier redaction is incomplete. AI scores and suggestions can be inaccurate and require human review.
- Scanned PDFs need OCR, which is not implemented.

10. Installation and Demonstration

10.1 Prerequisites

- Docker Desktop running Linux containers with engine API available.
- Terraform >= 1.5, Docker CLI, internet to pull images/install packages.
- No cloud account for the local lab; Groq API key only for real AI analysis.

10.2 Local run

```
$env:DOCKER_HOST = 'npipe:////./pipe/dockerDesktopLinuxEngine'
docker info
terraform -chdir=terraform init
terraform -chdir=terraform fmt -check
terraform -chdir=terraform validate
terraform -chdir=terraform apply
terraform -chdir=terraform output
Invoke-RestMethod http://127.0.0.1:5000/health
# Open http://127.0.0.1:8080/nagios4/ in a browser
terraform -chdir=terraform destroy
```

Configure GROQ_API_KEY in Render or a private local .env file. GitHub Actions image publishing separately requires DOCKER_USERNAME and DOCKER_TOKEN. Never include secret values in screenshots.

11. Conclusion

This project combines an AI-assisted resume review application with a coherent DevOps lab: Terraform-managed Docker resources, Ansible orchestration, a Puppet module with package/file/service resources, and a Nagios HTTP service check. The boundary between the local lab and Render avoids unsupported host-access claims. The remaining practical requirement is to run the lab against a healthy Docker Engine, capture genuine integration evidence, and complete the cover placeholders.

Appendix A. Rubric-to-Evidence Mapping

Evaluation criterion	Marks	Implementation / evidence
Problem selection, innovation & relevance	10	Resume-to-role matching is a relevant automation problem; scope and limits are explicit.
Architecture & configuration-management design	10	Docker lab integrates Terraform, Ansible, Puppet, and Nagios; Render remains separate.
Puppet implementation & configuration automation	15	Package, template-backed file, and service resources manage real Nagios configuration.
Nagios monitoring implementation	10	HTTP health command, host/service objects, thresholds, validation, and local UI.
Terraform Infrastructure as Code	15	Pinned provider, image/network/container resources, outputs, dependencies, and teardown.
Ansible automation & playbooks	15	Docker inventory/connection, Puppet apply, Nagios checks, and health assertion.
Tool integration / end-to-end automation	10	Terraform invokes Ansible; Ansible applies Puppet and validates Nagios and the app.
Testing, validation & technical evidence	5	Five app tests and Terraform validation passed; live lab evidence is still required.
Project report & documentation	5	This report maps implementation to the rubric and gives reproducible commands.
Viva & live demonstration	5	Appendix includes a demo sequence and design-question prompts.

Appendix B. Viva and Demonstration Questions

- Why separate Render and the local lab? Render abstracts away the host; the lab supplies disposable controllable containers.
- What does Terraform own? Images, isolated network, three containers, and dependency-triggered Ansible execution.
- What does Puppet own? Nagios plugin package, host/service object file, and monitoring service state.
- Why use Ansible? Inventory, target connections, repeatable orchestration, retries, and assertions.
- How is integration proven? Apply invokes Ansible; Ansible applies Puppet and validates Nagios and the app.
- What does /health prove? Process readiness only, not Groq API validity or end-to-end analysis.
- What are the risks? Docker socket control, demo credentials, inaccurate AI output, and incomplete redaction.

References

- Terraform Docker provider: https://registry.terraform.io/providers/kreuzwerker/docker/latest/docs
- Terraform language: https://developer.hashicorp.com/terraform/language
- Ansible Docker connection: https://docs.ansible.com/ansible/latest/collections/community/docker/docker_connection.html
- Ansible URI module: https://docs.ansible.com/ansible/latest/collections/ansible/builtin/uri_module.html
- Puppet resources: https://help.puppet.com/core/current/Content/PuppetCore/lang_resources.htm
- Nagios Core documentation: https://assets.nagios.com/downloads/nagioscore/docs/nagioscore/4/en/
- Nagios check_http: https://nagios-plugins.org/doc/man/check_http.html
- Docker documentation: https://docs.docker.com/
- Render documentation: https://render.com/docs